

CIS 5357

Computing for Data Analytics

Week 5

15 FEBRUARY 2023

Last week

- lists
- for loops
- while loops

Quick refresher

- How do you access an item in a list? indexing or slicing [x] [:x] [x:y] [x:]
- How would you convert a string to a list? .split()
- How would you convert a list to a string? .join()
- How do you add an item to a list? .append() .extend() .insert()
- How do you remove an item from a list? .remove() .pop() del .clear()
- How do you sum values in a list? sum()
- How do you count the values in a list? len() .count(x)
- How do you sort a list? .sort() .reverse() sorted(x) reversed(x)



Module 4

Practice

Using Python write code to iterate through every number in the list below to separate out even and odd numbers. Then return the number of odd and even numbers in the list.

```
practice_list = [12, 21, 33, 14, 57, 6, 17, 10, 11, 28, 3, 32, 2, 4]
```

Then test your code with another random list. Does it still work correctly?

Note this is not a graded assignment, but you should bring your ideas to the next class, where we'll discuss the different approaches everyone took.

A solution using counters

```
practice_list = [12, 21, 33, 14, 57, 6, 17, 10, 11, 28, 3, 32, 2, 4]

# define counter variables
evens = 0
odds = 0

# iterate through the list
for n in practice_list :
    if n % 2 == 0: # check if the number is even
        evens = evens+1
    else:
        odds = odds+1

# print the number of evens and odds
print(f"There are {evens} even numbers and {odds} odd numbers")
```

A solution using lists

```
practice_list = [12, 21, 33, 14, 57, 6, 17, 10, 11, 28, 3, 32, 2, 4]
```

```
#create a list of even numbers
```

```
evens = [n for n in practice_list if n % 2 == 0]
```

```
#create a list of odd numbers
```

```
odds = [n for n in practice_list if n % 2 != 0]
```

```
# print the number of evens and odds
```

```
print(f"There are {len(evens)} even numbers and {len(odds)} odd numbers.")
```

```
# print the even and odd numbers in order
```

```
print(f"The even numbers are: {sorted(evens)} and the odd numbers are: {sorted(odds)}.")
```

Another solution using lists

```
practice_list = [12, 21, 33, 14, 57, 6, 17, 10, 11, 28, 3, 32, 2, 4]

# define lists to store even and odd numbers
evens = []
odds = []

# iterate through the list
for n in practice_list :
    if n % 2 == 0: # check if the number is even
        evens.append(n)
    else:
        odds.append(n)

# print the number of evens and odds
print(f"There are {len(evens)} even numbers and {len(odds)} odd numbers.")

# print the even and odd numbers in order
print(f"The even numbers are: {sorted(evens)} and the odd numbers are: {sorted(odds)}.")
```

A shorter version

```
practice_list = [12, 21, 33, 14, 57, 6, 17, 10, 11, 28, 3, 32, 2, 4]

# define lists to store even and odd numbers
evens = []
odds = []

# iterate through the list
for n in practice_list:
    evens.append(n) if n % 2 == 0 else odds.append(n)

# print the number of evens and odds
print(f"There are {len(evens)} even numbers and {len(odds)} odd numbers.")

# print the even and odd numbers in order
print(f"The even numbers are: {sorted(evens)} and the odd numbers are: {sorted(odds)}.")
```

A shorter version

```
practice_list = [12, 21, 33, 14, 57, 6, 17, 10, 11, 28, 3, 32, 2, 4]
```

```
# define lists to store even and odd numbers
```

```
evens = []
```

```
odds = []
```

```
# iterate through the  
for n in practice_list  
    evens.append(n) if
```

What's missing in all of these examples?

```
# print the number of evens and odds
```

```
print(f"There are {len(evens)} even numbers and {len(odds)} odd numbers.")
```

```
# print the even and odd numbers in order
```

```
print(f"The even numbers are: {sorted(evens)} and the odd numbers are: {sorted(odds)}.")
```

Include exception handling

```
# define lists to store even and odd numbers
evens, odds = [], []

# Exception handling to check if the list contains only numerical values
try:
    # iterate through the list
    for n in list:
        evens.append(n) if n % 2 == 0 else odds.append(n)

    # print the number of evens and odds
    print(f"There are {len(evens)} even numbers and {len(odds)} odd numbers.")

except TypeError:
    print("The list contains non-numerical values")
```

This week

- functions
- common packages (libraries)
- web scraping

Share in chat

What Python function have
we already seen?

Python functions versus methods

In Python, methods and functions have similar purposes but differ in important ways.

- **Functions** are independent blocks of code that can be called from anywhere. For example, *print()*
- **Methods** are tied to objects or classes and need an object or class instance to be invoked. For example, *.upper()*

Download code to follow along

Module 5

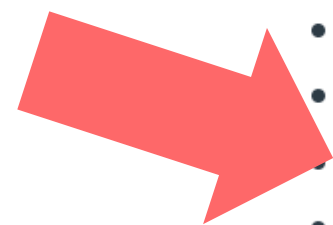
Read and View

Read

- Read the textbook chapter 4
- [Defining functions](#) ↗
- [More on defining functions](#) ↗

For reference

- [Python Standard Library](#) ↗
- <https://pypi.org/> ↗
- [Beautiful Soup library](#) ↗
- [Lecture practice code](#) ↓
- Lecture slides (coming soon)

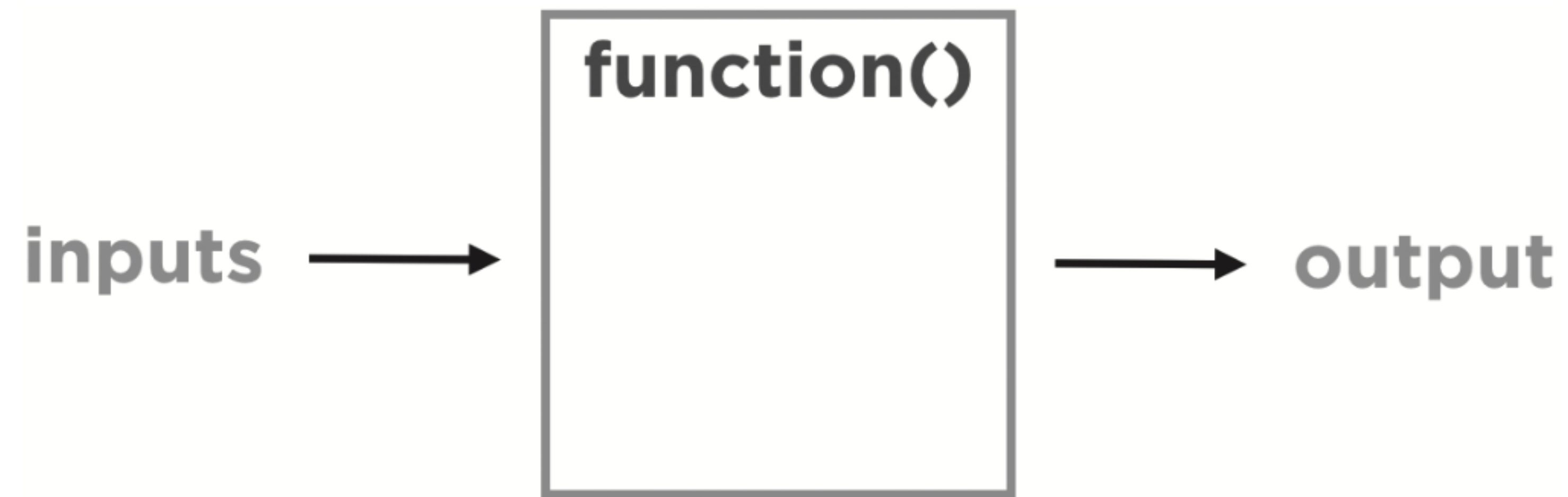


View

[Python Functions for Beginners](#) ↗

Functions

Snippets of code that you can reuse over and over



Functions

Snippets of code that you can reuse over and over

An example:

```
def average(numbers):  
    return sum(numbers) / len(numbers)
```

Functions

Snippets of code that you can reuse over and over

An example:

```
def average(numbers):  
    return sum(numbers) / len(numbers)
```



1. Start with **def** (this stands for define) and a **space**.

Functions

Snippets of code that you can reuse over and over

An example:

```
def average(numbers):  
    return sum(numbers) / len(numbers)
```

2. Then put the **function name**. Recommend lowercase words with underscores instead of spaces. Use this later to run (or “call”) the function.

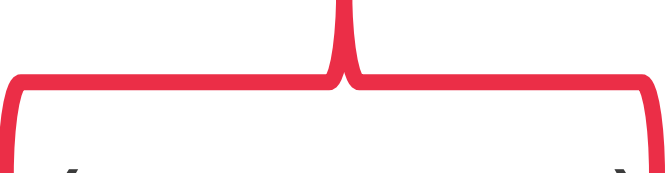
Functions

Snippets of code that you can reuse over and over

An example:

3. Then we need **parentheses**, which is where we can put any inputs (or **arguments**) that get passed in when the function is actually run later. These will end up being used like variables inside the function .

```
def average(numbers):  
    return sum(numbers) / len(numbers)
```



Functions

Snippets of code that you can reuse over and over

An example:

```
def average(numbers):  
    return sum(numbers) / len(numbers)
```

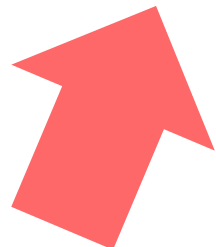
4. We finish the function definition line with a **:** (colon). Don't forget this.

Functions

Snippets of code that you can reuse over and over

An example:

```
def average(numbers):  
    return sum(numbers) / len(numbers)
```



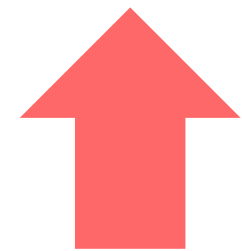
5. After we've named the function and set its arguments, we then have to **indent** all the code we want to put inside the function. Just like with if and for loops, Python looks at indentation to figure out what code is part of a function and what isn't.

Functions

Snippets of code that you can reuse over and over

An example:

```
def average(numbers):  
    return sum(numbers) / len(numbers)
```



6. The **return** keyword indicates what gets returned

Generic function structure

You can have multiple inputs and several lines of code in the body of a function

```
def function(input1, input2, ...):  
    # Do something with inputs  
    return ...
```

Calling a function

```
def average(numbers):  
    return sum(numbers) / len(numbers)
```

```
grades = [90, 85, 74]
```

```
average(grades)
```

Another function example

what does this function do?

```
def divisible_by(number, divisor):  
    if (number % divisor) == 0:  
        return True  
    else:  
        return False
```

How can we simplify this?

what does this function do?

```
def divisible_by(number, divisor):  
    if (number % divisor) == 0:  
        return True  
    else:  
        return False
```

A more simplified version

```
def divisible_by(number, divisor):  
    return (number % divisor) == 0
```

Less code is not always better

what's wrong here?

```
for x in range(100): print(x%3//2*'Fizz'+x%5//4*'Buzz' or x+1)
```

Labeling inputs

We have the option to explicitly label our function arguments when we pass them into a function:

```
divisible_by(number = 15, divisor = 3)
```

Labeling inputs

We have the option to explicitly label our function arguments when we pass them into a function:

```
divisible_by(number = 15, divisor = 3)
```

This can help clarify what each function input actually is. It also allows us to reorder the function arguments if we wanted to:

```
divisible_by(divisor = 3, number = 15)
```

Optional inputs

Labeling a function argument is sometimes necessary in cases in which a function argument has a ***default value***. When we define a function, we can give one of its arguments a default:

```
def future_value(present_value, rate, periods=1):  
    return round(present_value * (1 + rate) ** periods, 2)
```

Optional inputs

Labeling a function argument is sometimes necessary in cases in which a function argument has a ***default value***. When we define a function, we can give one of its arguments a default:

```
def future_value(present_value, rate, periods=1):  
    return round(present_value * (1 + rate) ** periods, 2)
```

With the addition of periods=1, we now have the option of omitting this argument when we run the function.

```
future_value(1000, .1)  
future_value(1000, .1, 3)
```

Questions?

Challenge 1: combine lists

Write a function that merges two lists into a new sorted list.

For example:

$[1,4,6]$ and $[2,3,5] \rightarrow [1,2,3,4,5,6]$

One possible solution

```
def merge_sort1(lst1, lst2):  
    combo_list = (lst1 + lst2)  
    combo_list.sort()  
    return combo_list
```

```
# test
```

```
x = [1,4,6]
```

```
y = [2,3,5]
```

```
merge_sort1(x,y)
```

Another possible solution

```
def merge_sort2(lst1, lst2):  
    lst1.extend(lst2)  
    return sorted(lst1)  
  
# test  
x = [1,4,6]  
y = [2,3,5]  
merge_sort2(x,y)
```

A simpler solution

```
def merge_sort_3(lst1, lst2):  
    return sorted(lst1 + lst2)
```

```
# test
```

```
x = [1,4,6]
```

```
y = [2,3,5]
```

```
merge_sort_3(x,y)
```

This week

- functions
- **common packages (libraries)**
- web scraping

Python packages

A **package** (sometimes also referred to in Python as a *library* or *module*) is a generic term used to describe a bunch of code that was designed to be used by many different scripts or applications.

Python packages

A **package** (sometimes also referred to in Python as a *library* or *module*) is a generic term used to describe a bunch of code that was designed to be used by many different scripts or applications.

Remember *happy_hour.py*

```
import random
```

```
bars = ["McSorley's Old Ale House", "Death & Company", "PDT"]
```

```
random_bar = random.choice(bars)
```

Importing packages

The general syntax for importing a package and then using one of the functions from that package follows:

```
import package  
package.function()
```

Importing packages

The general syntax for importing a package and then using one of the functions from that package follows:

```
import package  
package.function()
```

Note that running this exact code, however, will produce an error:

```
ModuleNotFoundError: No module named 'package'  
NameError: name 'package' is not defined
```

Package alias

It's possible to import a package and give it a **shorter alias** using `as`, like so:

```
import statistics as st
st.mean([1, 2, 3])
st.mean([12, 87, 10.5])
```

Import specific functions

If we know that there's only **one specific function** we want to use inside of a package, we can import only that function using from and import like this:

```
from statistics import mean  
mean([1, 2, 3])  
mean([12, 87, 10.5])
```

Note that we don't have to put the parentheses after mean in from statistics import mean

Python Standard Library

The set of built-in packages that comes with Python is collectively known as the *Python Standard Library*.

Python Standard Library

The set of built-in packages that comes with Python is collectively known as the *Python Standard Library*.

In the Read and View module, you'll find a link to a page listing a whole bunch of packages like *random*, *math*, and *statistics* that are available in the *Python Standard Library*.

The Python Standard Library

While [The Python Language Reference](#) describes the exact syntax and semantics of the Python language, this library reference manual describes the standard library that is distributed with Python. It also describes some of the optional components that are commonly included in Python distributions.

Python's standard library is very extensive, offering a wide range of facilities as indicated by the long table of contents listed below. The library contains built-in modules (written in C) that provide access to system functionality such as file I/O that would otherwise be inaccessible to Python programmers, as well as modules written in Python that provide standardized solutions for many problems that occur in everyday programming. Some of these modules are explicitly designed to encourage and enhance the portability of Python programs by abstracting away platform-specifics into platform-neutral APIs.

The Python installers for the Windows platform usually include the entire standard library and often also include many additional components. For Unix-like operating systems Python is normally provided as a collection of packages, so it may be necessary to use the packaging tools provided with the operating system to obtain some or all of the optional components.

In addition to the standard library, there is an active collection of hundreds of thousands of components (from individual programs and modules to packages and entire application development frameworks), available from the [Python Package Index](#).

- [Introduction](#)
 - [Notes on availability](#)
- [Built-in Functions](#)
- [Built-in Constants](#)
 - [Constants added by the site module](#)
- [Built-in Types](#)

Learn about the *random* package

- reprlib — Alternate repr() implementation
- enum — Support for enumerations
- graphlib — Functionality to operate with graph-like structures
- Numeric and Mathematical Modules
 - numbers — Numeric abstract base classes
 - math — Mathematical functions
 - cmath — Mathematical functions for complex numbers
 - decimal — Decimal fixed point and floating point arithmetic
 - fractions — Rational numbers
 - random — Generate pseudo-random numbers
 - statistics — Mathematical statistics functions
- Functional Programming Modules
 - itertools — Functions creating iterators for efficient looping
 - functools — Higher-order functions and operations on callable objects
 - operator — Standard operators as functions
- File and Directory Access
 - pathlib — Object-oriented filesystem paths

Learn about the *random* package

- `reprlib` — Alternate `repr()` implementation
- `enum` — Support for enumerations
- `graphlib` — Functionality to operate with graph-like structures
- **Numeric and Mathematical Modules**
 - `numbers` — Numeric abstract base classes
 - `math` — Mathematical functions
 - `cmath` — Mathematical functions for complex numbers
 - `decimal` — Decimal fixed point and floating point arithmetic
 - `fractions` — Rational numbers
 - **`random` — Generate pseudo-random numbers**
 - `statistics` — Mathematical statistics functions
- **Functional Programming Modules**
 - `itertools` — Functions creating iterators for efficient looping
 - `functools` — Higher-order functions and operations on callable objects
 - `operator` — Standard operators as functions
- **File and Directory Access**
 - `pathlib` — Object-oriented filesystem paths

`random` — Generate pseudo-random numbers

Source code: [Lib/random.py](https://lib.python.org/random.py)

This module implements pseudo-random number generators for various distributions.

For integers, there is uniform selection from a range. For sequences, there is uniform selection of a random element, a function to generate a random permutation of a list in-place, and a function for random sampling without replacement.

On the real line, there are functions to compute uniform, normal (Gaussian), lognormal, negative exponential, gamma, and beta distributions. For generating distributions of angles, the von Mises distribution is available.

Almost all module functions depend on the basic function `random()`, which generates a random float uniformly in the half-open range $0.0 \leq X < 1.0$. Python uses the Mersenne Twister as the core generator. It produces 53-bit precision floats and has a period of $2^{19937}-1$. The underlying implementation in C is both fast and threadsafe. The Mersenne Twister is one of the most extensively tested random number generators in existence. However, being completely deterministic, it is not suitable for all purposes, and is completely unsuitable for cryptographic purposes.

The functions supplied by this module are actually bound methods of a hidden instance of the `random.Random` class. You can instantiate your own instances of `Random` to get generators that don't share state.

Learn about the *random* package

In the documentation for the random package, you'll find a bunch of other interesting functions all related to randomness, such as the following:

- **random()**—generates a random number between zero and one
- **shuffle()**—shuffles around a list randomly
- **sample()**—takes a random sampling of numbers from a list
- **uniform(a, b)**—generates a random number between any 2 numbers

3rd party packages

On top of the built-in packages that come included with Python, hundreds of thousands of third-party packages are written and made available by other coders online.

3rd party packages

On top of the built-in packages that come included with Python, hundreds of thousands of third-party packages are written and made available by other coders online.

Python comes with the **command line command** *pip*, which makes it super easy to get new Python packages from the web.

```
% pip install pandas
```

3rd party packages

On top of the built-in packages that come included with Python, hundreds of thousands of third-party packages are written and made available by other coders online.

Python comes with the **command line command** *pip*, which makes it super easy to get new Python packages from the web.

```
% pip install pandas
```

This code would go to **pypi.org** and download the *pandas* package to our computer. Note that this *pip* install line should be **run directly in your command line**, not in Python.

Common libraries for data analysis

Pandas

NumPy

SciPy

SciKit-Learn

Matplotlib

Seaborn

Plotly

Challenge 2: explore

Explore the *Python Standard Library* documentation and pick one function each from the *statistics*, *math*, and *datetime* packages to import and learn how to use.

Challenge 2: explore

Explore the *Python Standard Library* documentation and pick one function each from the *statistics*, *math*, and *datetime* packages to import and learn how to use.

Share a brief comment in *chat* explaining how one of the functions you chose works.

This week

- functions
- common packages (libraries)
- **web scraping**

Web scraping

- Web scraping is the process of extracting data from websites.
- Useful to extract public data from websites.
- In Python, this can be achieved using several libraries.

Scrape responsibly

- Only scrape public information
- Respect website policies
- Consider legal and ethical aspects
- Some sites block scraping
- Dynamic content consideration

Web scraping libraries

Library	Key Features
Beautiful Soup	HTML/XML parsing; easy navigation
Requests	custom headers; session objects
Selenium	useful for dynamic content (not covered in this course)

Web scraping libraries

Library	Key Features
Beautiful Soup	HTML/XML parsing; easy navigation
Requests	custom headers; session objects

```
import requests  
from bs4 import BeautifulSoup
```

Identify the url

quotes.toscrape.com

Quotes to Scrape

"The world as we have created it is a process of our thinking. It cannot be changed without changing our thinking."

by [Albert Einstein](#) (about)

Tags: [change](#) [deep-thoughts](#) [thinking](#) [world](#)

"It is our choices, Harry, that show what we truly are, far more than our abilities."

by [J.K. Rowling](#) (about)

```
# send HTTP request
response = requests.get('https://quotes.toscrape.com/') # target url

# parse the HTML content
soup = BeautifulSoup(response.content, 'html.parser')
```

Inspect code to find tags

quotes.toscrape.com

Quotes to Scrape

"The world as we have created it is a process of our thinking. It cannot be changed without changing our thinking."
by [Albert Einstein](#) (about)
Tags: [change](#) [deep-thoughts](#) [thinking](#) [world](#)

"It is our choices, Harry, that show what we truly are, far more than our abilities."
by [J.K. Rowling](#) (about)
Tags: [abilities](#) [choices](#)

"There are only two ways to live your life. One is as though nothing is a miracle. The other is as though everything is a miracle."
by [Albert Einstein](#) (about)
Tags: [inspirational](#) [life](#) [live](#) [miracle](#) [miracles](#)

Top Te
insp

- Look Up "without"
- Copy
- Copy Link to Highlight
- Search Google for "without"
- Print...
- Translate Selection to English
- Open in Reading Mode **NEW**
- LastPass >
- Inspect**
- Speech >
- Services >

Inspect code to find tags

Quotes to Scrape

div.quote 720 x 134.13

"The world as we have created it is a process of our thinking. It cannot be changed without changing our thinking."

by [Albert Einstein](#) (about)

Tags: [change](#) [deep-thoughts](#) [thinking](#) [world](#)

"It is our choices, Harry, that show what we truly are, far more than our abilities."

by [J.K. Rowling](#) (about)

Tags: [abilities](#) [choices](#)

Login

```
<!DOCTYPE html>
<html lang="en">
  <head> ... </head>
  <body>
    <div class="container">
      ::before
      <div class="row header-box"> ... </div>
      <div class="row">
        ::before
        <div class="col-md-8">
          <div class="quote" itemscope itemtype="http://schema.org/CreativeWork">
            <span class="text" itemprop="text"> ... </span> == $0
            <span> ... </span>
            <div class="tags"> ... </div>
          </div>
          <div class="quote" itemscope itemtype="http://schema.org/CreativeWork"> ... </div>
          <div class="quote" itemscope itemtype="http://schema.org/CreativeWork"> ... </div>
          <div class="quote" itemscope itemtype="http://schema.org/CreativeWork"> ... </div>
          <div class="quote" itemscope itemtype="http://schema.org/CreativeWork"> ... </div>
          <div class="quote" itemscope itemtype="http://schema.org/CreativeWork"> ... </div>
          <div class="quote" itemscope itemtype="http://schema.org/CreativeWork"> ... </div>
        </div>
      </div>
    </div>
  </body>
</html>
```

BeautifulSoup .find_all method

The *.find_all()* method allows you to find elements on the page by class name, id, or any other element attribute using the attrs parameter.

```
# tag + any attribute – adjust based on the url  
quotes = soup.find_all('span', attrs={'class': 'text'})
```

```
# tag + class name example  
quotes = soup.find_all('span', class_='text')
```

```
# tag + id  
variable = soup.find_all('tag', id='id_name')
```

Print extracted data

```
# loop through to print extracted data  
for q in quotes:  
    print(q.text)
```

PROBLEMS

3

OUTPUT

DEBUG CONSOLE

TERMINAL

Python + v [] [] ... ^ x

"The world as we have created it is a process of our thinking. It cannot be changed without changing our thinking."

"It is our choices, Harry, that show what we truly are, far more than our abilities."

"There are only two ways to live your life. One is as though nothing is a miracle. The other is as though everything is a miracle."

"The person, be it gentleman or lady, who has not pleasure in a good novel, must be intolerably stupid."

"Imperfection is beauty, madness is genius and it's better to be absolutely ridiculous than absolutely boring."

Challenge 3: print quote & author

Extract and print both the quote along with the author's name from the url: **<https://quotes.toscrape.com/>**

One possible solution

```
import requests
from bs4 import BeautifulSoup

# send HTTP request
response = requests.get('https://quotes.toscrape.com/')

# check if the request was successful
if response.status_code == 200:
    # parse the HTML content
    soup = BeautifulSoup(response.content, 'html.parser')
    # extract data
    quotes = soup.find_all('span', attrs={'class': 'text'})
    authors = soup.find_all('small', attrs={'class': 'author'})
    # loop through to print extracted data
    for i in range(len(quotes)):
        print(quotes[i].text + "-" + authors[i].text)
else:
    print('Failed to retrieve the webpage')
```

Zip is also an option

```
import requests
from bs4 import BeautifulSoup

# send HTTP request
response = requests.get('https://quotes.toscrape.com/')

# check if the request was successful
if response.status_code == 200:
    # parse the HTML content
    soup = BeautifulSoup(response.content, 'html.parser')
    # extract data
    quotes = soup.find_all('span', attrs={'class': 'text'})
    authors = soup.find_all('small', attrs={'class': 'author'})
    # loop through to print extracted data
    for q, a in zip(quotes, authors):
        print(q.text + "-" + a.text)
else:
    print('Failed to retrieve the webpage')
```

Findall() with tag + class also works

```
import requests
from bs4 import BeautifulSoup

# send HTTP request
response = requests.get('https://quotes.toscrape.com/')

# check if the request was successful
if response.status_code == 200:
    # parse the HTML content
    soup = BeautifulSoup(response.content, 'html.parser')
    # extract data
    quotes = soup.find_all('span', class_='text')
    authors = soup.find_all('small', class_='author')
    # loop through to print extracted data
    for q, a in zip(quotes, authors):
        print(q.text + "-" + a.text)
else:
    print('Failed to retrieve the webpage')
```

Recap

- **Functions** are snippets of code that you can reuse over and over
- A **package** is a generic term used to describe a bunch of code that was designed to be used by many different scripts or applications
- The **Python Standard Library** is a set of built-in packages that comes with Python
- **3rd party packages** from other coders can also be used in Python code and loaded using the **command line command *pip***
- **Web scraping** is the process of extracting data from websites.

Share in chat

What Python function could
you write for your job or
personal use?

Next week

Coding assignment due by next class

Module 5

Coding assignment

Instructions

Imagine you are a marketing analyst, and your boss has asked you to analyze the effectiveness of marketing campaigns. They want you to evaluate each campaign's reach, engagement rate, and conversion rate to recommend whether to continue, modify, or discontinue a campaign based on its performance. Use Python and the requirements below to fulfill this request.

- Continue with any marketing campaign that has engagement rate above 15% or conversion rate above 5%.
- Any campaign with a conversion rate below 1% or reach below 3,000 should be discontinued, regardless of the engagement rate.
- Any campaigns not meeting the above criteria should be flagged for review and possible modification.

Below is a small sample of the marketing campaign performance data available to test your code.

Name	Reach	Engagement	Conversion
Recommendations	15,234	1,451	523
Product Highlights	2,030	750	35
Monthly Newsletter	25,031	1,120	2,123
Social Contest	230,981	1,250	1,124

Topic for next week: dictionaries